

Comparison Report

IIoT Sensor Network

Monica Joya, Cassy Cormier, Kolapo Mogaji, Sufyan Rafiq
ITAI 3377: IoT & Edge Computing
Lab 04: Conceptual Design of an IIoT Sensor Network
February 18, 2026

1. Introduction

This report presents a comparative analysis of three Industrial Internet of Things (IIoT) communication protocols: MQTT, CoAP, and OPC UA. Each protocol was implemented as a sensor simulation generating randomized temperature and humidity data. The goal was to evaluate how each protocol handles data transmission in an industrial IoT context, identify their strengths and limitations, and understand which scenarios each protocol is best suited for.

Throughout the process of building and troubleshooting each simulation, we consulted the official documentation for each protocol to develop a deeper understanding of how they work beyond what was covered in the lab instructions. Reviewing the MQTT v5.0 specification, the CoAP RFC 7252, and the OPC UA documentation from the OPC Foundation gave us valuable context on the design decisions behind each protocol and helped us make more informed observations during testing.

2. Protocol Overviews

2.1 MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight publish/subscribe messaging protocol designed for constrained devices and low-bandwidth networks. It operates over TCP and relies on a central broker (in our case, Mosquitto) to route messages between publishers and subscribers. In our simulation, the sensor script published JSON payloads containing temperature and humidity data to the *sensor/data* topic every second, while a separate visualization script subscribed to the same topic and plotted the data in real time.

2.2 CoAP (Constrained Application Protocol)

CoAP is a RESTful protocol designed for resource-constrained IoT devices. It uses UDP as its transport layer and follows a request/response model similar to HTTP, but with significantly lower overhead. Our simulation sent POST requests with sensor data payloads to a CoAP server endpoint. Notably, the original lab instructions only included a CoAP client script without a corresponding server. We identified this gap and created a simple CoAP server to receive and display the incoming sensor data, enabling a complete end-to-end demonstration.

2.3 OPC UA (Open Platform Communications Unified Architecture)

OPC UA is a platform-independent, service-oriented architecture designed for industrial automation and interoperability. Unlike MQTT and CoAP, OPC UA is a full-featured framework that includes built-in security, data modeling, and discovery. Our simulation created an OPC UA server that exposed temperature and humidity as structured variables under a defined

namespace, updating their values every second. The server was self-contained within a single script, requiring no external broker or additional server setup.

3. Protocol Comparison

The following table summarizes the key differences observed across the three protocols during our simulation:

Feature	MQTT	CoAP	OPC UA
Transport Layer	TCP	UDP	TCP
Architecture	Publish / Subscribe	Request / Response	Client / Server
Broker/Server	Requires external broker (Mosquitto)	Requires separate server (not provided in lab)	Self-contained server in script
Data Format	JSON payload (flexible)	JSON payload (flexible)	Structured variables with namespace
Overhead	Very low (2-byte header)	Low (4-byte header)	Higher (complex encoding)
Best Use Case	Telemetry, remote monitoring, constrained networks	Low-power devices, smart home, sensor networks	Industrial automation, SCADA, manufacturing
Setup Complexity	Moderate (broker installation required)	Moderate (server needed separately)	Low (self-contained)
Real-Time Visualization	Yes (via subscriber + Matplotlib)	Server-side logging	Terminal output

4. Key Observations

4.1 MQTT

- MQTT was the most straightforward to set up for real-time data visualization. The publish/subscribe model made it easy to decouple the data producer from the consumer.
- The Mosquitto broker handled message routing reliably, and the subscriber received data with no noticeable delay.
- The real-time Matplotlib chart showed temperature oscillating between 20–25°C (blue line) and humidity between 30–50% (orange line), updating every second with a rolling window of 100 data points.

4.2 CoAP

- The CoAP client initially returned a "Connection refused" error (Errno 61) because no server was provided in the lab instructions. This highlighted a key architectural difference: unlike MQTT which had Mosquitto, and OPC UA which bundles its own server, CoAP required us to build a separate server.

- After creating a CoAP server, the client successfully sent POST requests and received "2.04 Changed" response codes, which is CoAP's equivalent of HTTP's 200 OK.
- CoAP's use of UDP makes it inherently lighter than TCP-based protocols, but it lacks built-in reliability guarantees without additional confirmable message handling.

4.3 OPC UA

- OPC UA was the easiest to run because the server was self-contained within the Python script. No external broker or server installation was needed.
- The protocol uses structured data modeling with namespaces and typed variables, which is more rigid than MQTT's or CoAP's flexible JSON payloads but provides better interoperability in industrial environments.
- The terminal output displayed temperature and humidity values updating every second, confirming the server was correctly writing to its address space.

5. Conclusion

Each protocol serves a distinct purpose within the IIoT ecosystem. MQTT excels in lightweight telemetry and real-time data streaming, making it ideal for remote monitoring scenarios where bandwidth is limited. CoAP is well-suited for constrained devices that need RESTful communication over UDP, though it requires careful server-side planning. OPC UA provides the most comprehensive solution for industrial automation, with built-in data modeling and security, at the cost of higher complexity and overhead.

In practice, a modern IIoT deployment might use a combination of these protocols: MQTT for sensor telemetry at the edge, CoAP for battery-powered devices, and OPC UA for plant-floor systems that require standardized data exchange. Understanding the trade-offs between simplicity, overhead, and capability is essential for designing effective industrial IoT architectures.

6. References

Eclipse Foundation. (n.d.). *Eclipse Mosquitto: An open source MQTT broker*.
<https://mosquitto.org/>

Eclipse Foundation. (n.d.). *Paho MQTT Python client library*.
<https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html>

OASIS. (2019). *MQTT version 5.0: OASIS standard*.
<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

OPC Foundation. (n.d.). *OPC Unified Architecture specification*.
<https://opcfoundation.org/about/opc-technologies/opc-ua/>

Shelby, Z., Hartke, K., & Bormann, C. (2014). *The Constrained Application Protocol (CoAP)* (RFC 7252). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7252>

aiocoap contributors. (n.d.). *aiocoap: Python CoAP library*. <https://aiocoap.readthedocs.io/>

FreeOpcUa contributors. (n.d.). *opcua-asyncio: OPC UA library for Python*.
<https://github.com/FreeOpcUa/opcua-asyncio>